

CSA1709 - Artificial Intelligence Implementation Coding

Python Files:

[app.py](#):

```
from flask import Flask, render_template, request, jsonify
from database import get_connection, create_tables
from solver import generate_timetable
```

```
app = Flask(__name__)
```

```
create_tables()
```

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html")
```

```
@app.route("/add_subject", methods=["POST"])
```

```
def add_subject():
```

```
    data = request.json
```

```
    connection = get_connection()
```

```
    cursor = connection.cursor()
```

```
    cursor.execute("""
```

```
        INSERT INTO subjects
```

```
        (code, name, faculty, hours, students)
```

```
        VALUES (?, ?, ?, ?, ?)
```

```
    """, (
```

```
        data["code"],
```

```
        data["name"],
```

```
        data["faculty"],
```

```
        data["hours"],
```

```
        data["students"]
```

```
    ))
```

```
    connection.commit()
```

```
connection.close()
```

```
return jsonify({"success": True})
```

```
@app.route("/add_faculty", methods=["POST"])  
def add_faculty():
```

```
    data = request.json
```

```
    connection = get_connection()
```

```
    cursor = connection.cursor()
```

```
    cursor.execute("""  
        INSERT INTO faculty  
        (name, availability)  
        VALUES (?, ?)  
    """, (  
        data["name"],  
        data["availability"]  
    ))
```

```
    connection.commit()
```

```
    connection.close()
```

```
    return jsonify({"success": True})
```

```
@app.route("/add_room", methods=["POST"])  
def add_room():
```

```
    data = request.json
```

```
    connection = get_connection()
```

```
    cursor = connection.cursor()
```

```
    cursor.execute("""  
        INSERT INTO rooms  
        (room_number, capacity, room_type)  
        VALUES (?, ?, ?)
```

```
""", (
    data["room"],
    data["capacity"],
    data["type"]
))

connection.commit()
connection.close()

return jsonify({"success": True})
```

```
@app.route("/data")
def get_data():

    connection = get_connection()

    subjects = connection.execute(
        "SELECT * FROM subjects"
    ).fetchall()

    faculty = connection.execute(
        "SELECT * FROM faculty"
    ).fetchall()

    rooms = connection.execute(
        "SELECT * FROM rooms"
    ).fetchall()

    connection.close()

    return jsonify({
        "subjects": [dict(row) for row in subjects],
        "faculty": [dict(row) for row in faculty],
        "rooms": [dict(row) for row in rooms]
    })
@app.route("/generate", methods=["POST"])
def generate():

    result = generate_timetable()
```

```
    return jsonify(result)

if __name__ == "__main__":
    app.run(debug=True)
```

database.py:

```
import sqlite3
```

```
DATABASE = "timetable.db"
```

```
def get_connection():
    connection = sqlite3.connect(DATABASE)
    connection.row_factory = sqlite3.Row
    return connection
```

```
def create_tables():
```

```
    connection = get_connection()
    cursor = connection.cursor()
```

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS subjects (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            code TEXT NOT NULL,
            name TEXT NOT NULL,
            faculty TEXT NOT NULL,
            hours INTEGER NOT NULL,
            students INTEGER NOT NULL
        )
    """)
```

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS faculty (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
```

```
        availability TEXT NOT NULL
    )
    """
```

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS rooms (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        room_number TEXT NOT NULL,
        capacity INTEGER NOT NULL,
        room_type TEXT NOT NULL
    )
    """)
```

```
connection.commit()
connection.close()
```

[solver.py:](#)

```
from ortools.sat.python import cp_model
from database import get_connection
import time
```

```
DAYS = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday"
]
```

```
PERIODS = [
    "9-10",
    "10-11",
    "11-12",
    "1-2",
    "2-3"
]
```

```

def generate_timetable():

    start_time = time.time()

    connection = get_connection()

    subjects = connection.execute(
        "SELECT * FROM subjects"
    ).fetchall()

    rooms = connection.execute(
        "SELECT * FROM rooms"
    ).fetchall()

    connection.close()

    if not subjects:
        return {
            "success": False,
            "message": "No subjects found. Please add subjects first."
        }

    if not rooms:
        return {
            "success": False,
            "message": "No rooms found. Please add rooms first."
        }

    model = cp_model.CpModel()

    total_slots = len(DAYS) * len(PERIODS)

    assignments = {}

    for subject in subjects:

        for hour in range(subject["hours"]):

            for day in range(len(DAYS)):

```

```

for period in range(len(PERIODS)):

    for room in range(len(rooms)):

        key = (
            subject["id"],
            hour,
            day,
            period,
            room
        )

        assignments[key] = model.NewBoolVar(
            f'subject_{subject['id']}_hour_{hour}_day_{day}_period_{period}_room_{room}'
        )

```

```

# Constraint 1:
# Each required subject hour must be assigned exactly once.

```

```

for subject in subjects:

    for hour in range(subject["hours"]):

        variables = []

        for day in range(len(DAYS)):

            for period in range(len(PERIODS)):

                for room in range(len(rooms)):

                    key = (
                        subject["id"],
                        hour,
                        day,
                        period,
                        room
                    )

```

```

    )

    variables.append(assignments[key])

model.AddExactlyOne(variables)

# Constraint 2:
# A room cannot have two classes at the same time.

for day in range(len(DAYS)):

    for period in range(len(PERIODS)):

        for room in range(len(rooms)):

            variables = []

            for subject in subjects:

                for hour in range(subject["hours"]):

                    key = (
                        subject["id"],
                        hour,
                        day,
                        period,
                        room
                    )

                    variables.append(assignments[key])

            model.Add(sum(variables) <= 1)

# Constraint 3:
# A faculty member cannot teach two classes
# at the same time.

faculty_names = {}

```

```

for subject in subjects:
    faculty_names.setdefault(
        subject["faculty"],
        []
    ).append(subject["id"])

for faculty, subject_ids in faculty_names.items():

    for day in range(len(DAYS)):

        for period in range(len(PERIODS)):

            variables = []

            for subject in subjects:

                if subject["id"] not in subject_ids:
                    continue

                for hour in range(subject["hours"]):

                    for room in range(len(rooms)):

                        key = (
                            subject["id"],
                            hour,
                            day,
                            period,
                            room
                        )

                        variables.append(assignments[key])

            model.Add(sum(variables) <= 1)

# Constraint 4:
# Room capacity must be sufficient.

```

```

for subject in subjects:

    for hour in range(subject["hours"]):

        for day in range(len(DAYS)):

            for period in range(len(PERIODS)):

                for room_index, room in enumerate(rooms):

                    if room["capacity"] < subject["students"]:

                        key = (
                            subject["id"],
                            hour,
                            day,
                            period,
                            room_index
                        )

                        model.Add(assignments[key] == 0)


# Create solver

solver = cp_model.CpSolver()

solver.parameters.max_time_in_seconds = 10

solver.parameters.num_search_workers = 8


status = solver.Solve(model)


if status in (
    cp_model.OPTIMAL,
    cp_model.FEASIBLE
):

```

```

timetable = []

for subject in subjects:

    for hour in range(subject["hours"]):

        for day in range(len(DAYS)):

            for period in range(len(PERIODS)):

                for room_index, room in enumerate(rooms):

                    key = (
                        subject["id"],
                        hour,
                        day,
                        period,
                        room_index
                    )

                    if solver.Value(assignments[key]) == 1:

                        timetable.append( {

                            "subject": subject["name"],

                            "code": subject["code"],

                            "faculty": subject["faculty"],

                            "day": DAYS[day],

                            "period": PERIODS[period],

                            "room": room["room_number"],

                            "students": subject["students"]

                        })

```

```
generation_time = round(  
    time.time() - start_time,  
    3  
)
```

```
return {  
  
    "success": True,  
  
    "timetable": timetable,  
  
    "generation_time": generation_time,  
  
    "status": "OPTIMAL"  
    if status == cp_model.OPTIMAL  
    else "FEASIBLE"  
  
}
```

```
return {  
  
    "success": False,  
  
    "message":  
    "No conflict-free timetable could be generated."  
  
}
```

Index.html

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AI Timetable Scheduler</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>

<body>

<div class="sidebar">

  ``
<div class="logo">
  <div class="logo-icon"><img alt="AI Scheduler logo" data-bbox="328 433 353 451"/></div>
  <div>
    <h2>AI Scheduler</h2>
    <span>Smart Timetabling</span>
  </div>
</div>

<div class="menu">

  <a class="menu-item active" href="#">
    <span><img alt="Dashboard icon" data-bbox="215 633 240 651"/></span>
    Dashboard
  </a>

  <a class="menu-item" href="#data">
    <span><img alt="Data Management icon" data-bbox="215 733 240 751"/></span>
    Data Management
  </a>

  <a class="menu-item" href="#constraints">
    <span><img alt="Constraints icon" data-bbox="215 833 240 851"/></span>
    Constraints
  </a>
```

```
<a class="menu-item" href="#generate">
  <span>⚡</span>
  Generate
</a>
```

```
<a class="menu-item" href="#report">
  <span>📊</span>
  Reports
</a>
```

```
</div>
```

```
<div class="sidebar-bottom">
  <div class="status-dot"></div>
  System Ready
</div>
...
```

```
</div>
```

```
<div class="main">
```

```
...
```

```
<div class="topbar">
```

```
<div>
  <p class="welcome">WELCOME BACK 🖐️</p>
  <h1>AI Timetable Dashboard</h1>
</div>
```

```
<div class="top-status">
  <span class="green-dot"></span>
  Solver Ready
</div>
```

```
</div>
```

```
<section class="hero">
```

```
<div class="hero-content">
```

```
<div class="ai-badge">
```

```
  ✨ AI POWERED
```

```
</div>
```

```
<h1>
```

```
  Generate Smarter,<br>
```

```
  <span>Conflict-Free Timetables.</span>
```

```
</h1>
```

```
<p>
```

```
  Automatically create optimized academic timetables  
  using Constraint Satisfaction Problem solving,  
  intelligent heuristics and AI optimization.
```

```
</p>
```

```
<button class="generate-btn" onclick="generateTimetable()">
```

```
  ⚡ Generate Timetable
```

```
</button>
```

```
</div>
```

```
<div class="hero-visual">
```

```
<div class="brain-circle">
```



```
</div>
```

```
<div class="floating-card card-one">
```

```
  ✓ No Faculty Conflicts
```

```
</div>
```

```
<div class="floating-card card-two">
```

```
  ✓ Room Optimized
```

```
</div>
```

```
<div class="floating-card card-three">
```

```
  ✓ 100% Constraint Satisfaction
```

</div>

</div>

</section>

<section class="stats">

<div class="stat-card">

<div class="stat-icon purple"> </div>

<div>

<p>Subjects</p>

<h2 id="subjectCount">6</h2>

<small>Active subjects</small>

</div>

</div>

<div class="stat-card">

<div class="stat-icon blue"> </div>

<div>

<p>Faculty</p>

<h2 id="facultyCount">5</h2>

<small>Teaching staff</small>

</div>

</div>

<div class="stat-card">

<div class="stat-icon orange"> </div>

<div>

```
<p>Rooms</p>
<h2 id="roomCount">3</h2>
<small>Available rooms</small>
</div>
```

```
</div>
```

```
<div class="stat-card">
```

```
<div class="stat-icon green"> </div>
```

```
<div>
  <p>Time Slots</p>
  <h2 id="slotCount">25</h2>
  <small>Available slots</small>
</div>
```

```
</div>
```

```
</section>
```

```
<section class="dashboard-grid">
```

```
<div class="panel">
```

```
<div class="panel-header">
```

```
<div>
  <h2>Constraint Satisfaction</h2>
  <p>Current scheduling performance</p>
</div>
```

```
<span class="success-badge">Excellent</span>
```

```
</div>
```

```
<div class="progress-container">
```

```
<div class="progress-info">
  <span>Overall Satisfaction</span>
  <strong id="percentage">100%</strong>
</div>
```

```
<div class="progress-bar">
  <div class="progress" id="progress"></div>
</div>
```

```
</div>
```

```
<div class="constraint-list">
```

```
<div>
  <span>✓</span>
  Faculty conflicts
  <strong>0</strong>
</div>
```

```
<div>
  <span>✓</span>
  Room conflicts
  <strong>0</strong>
</div>
```

```
<div>
  <span>✓</span>
  Availability conflicts
  <strong>0</strong>
</div>
```

```
<div>
  <span>✓</span>
  Capacity conflicts
  <strong>0</strong>
</div>
```

```
</div>
```

```
</div>
```

<div class="panel system-panel">

<div class="panel-header">

<div>

<h2>AI Solver</h2>

<p>Optimization engine</p>

</div>

</div>

<div class="solver-status">

<div class="solver-icon">



</div>

<div>

<h3>CP-SAT Solver</h3>

<p>Ready to generate solution</p>

</div>

READY

</div>

<div class="solver-info">

<div>

Algorithm

Constraint Programming

</div>

<div>

Search

Optimized

</div>

</div>

</div>

</section>

<section class="data-section" id="data">

<div class="section-title">

<div>

<h2> Data Management</h2>

<p>Add the information required for timetable generation</p>

</div>

</div>

<div class="data-grid">

<div class="data-card">

<h3> Add Subject</h3>

<input type="text" id="subjectCode" placeholder="Subject Code">

<input type="text" id="subjectName" placeholder="Subject Name">

<input type="text" id="subjectFaculty" placeholder="Faculty Name">

<div class="input-row">

<input type="number" id="subjectHours" placeholder="Hours / Week">

<input type="number" id="studentCount" placeholder="Students">

</div>

<button onclick="addSubject()">+ Add Subject</button>

</div>

<div class="data-card">

<h3> Add Faculty</h3>

<input type="text" id="facultyName" placeholder="Faculty Name">

<select id="facultyAvailability">

```
<option value="All Days">Available All Days</option>
<option value="Morning">Morning Only</option>
<option value="Afternoon">Afternoon Only</option>
</select>
```

```
<button onclick="addFaculty()">+ Add Faculty</button>
```

```
</div>
```

```
<div class="data-card">
```

```
<h3>🏠 Add Room</h3>
```

```
<input type="text" id="roomNumber" placeholder="Room Number">
```

```
<input type="number" id="roomCapacity" placeholder="Capacity">
```

```
<select id="roomType">
  <option value="Classroom">Classroom</option>
  <option value="Laboratory">Laboratory</option>
</select>
```

```
<button onclick="addRoom()">+ Add Room</button>
```

```
</div>
```

```
<div class="data-card">
```

```
<h3>🕒 Time Slots</h3>
```

```
<p class="slot-info">
  The system automatically provides 25
  available timetable slots.
</p>
```

```
<div class="slot-summary">
  <div>
    <strong>5</strong>
    <span>Days</span>
  </div>
```

```
<div>
  <strong>5</strong>
  <span>Periods / Day</span>
</div>
```

```
<div>
  <strong>25</strong>
  <span>Total Slots</span>
</div>
</div>
```

```
<button onclick="showMessage('25 time slots are available')">
  ✓ Use Available Slots
</button>
```

```
</div>
```

```
</div>
```

```
<div class="table-panel">
```

```
<div class="table-header">
  <div>
    <h3>Added Subjects</h3>
    <p>Subjects currently available for scheduling</p>
  </div>
</div>
```

```
<table>
```

```
<thead>
  <tr>
    <th>Code</th>
    <th>Subject</th>
    <th>Faculty</th>
    <th>Hours</th>
    <th>Students</th>
  </tr>
```

</thead>

<tbody id="subjectTable">

</tbody>

</table>

</div>

</section>

<section class="quick-actions" id="generate">

<div>

<h2>Quick Actions</h2>

<p>Manage your timetable system</p>

</div>

<div class="actions">

<button onclick="showMessage('Data Management')">



Manage Data

</button>

<button onclick="generateTimetable()">



Generate Timetable

</button>

<button onclick="showMessage('Conflict Report')">



View Conflicts

</button>

</div>

</section>

<section class="constraints-section" id="constraints">

<div class="section-title">

<div>

<h2>⚙️ Scheduling Constraints</h2>

<p>Rules used by the AI constraint satisfaction solver</p>

</div>

Active

</div>

<div class="constraint-grid">

<div class="constraint-card">

<div class="constraint-icon">👤</div>

<div>

<h3>Faculty Conflict</h3>

<p>

A faculty member cannot teach
two classes at the same time.

</p>

</div>

✔

</div>

<div class="constraint-card">

<div class="constraint-icon">🏠</div>

<div>

<h3>Room Conflict</h3>

<p>

A room cannot be assigned to
multiple classes simultaneously.

</p>

</div>

✓

</div>

<div class="constraint-card">

<div class="constraint-icon">👥</div>

<div>

<h3>Room Capacity</h3>

<p>

The assigned room must have
sufficient capacity for students.

</p>

</div>

✓

</div>

<div class="constraint-card">

<div class="constraint-icon">🕒</div>

<div>

<h3>Time Availability</h3>

<p>

Classes are assigned only
within available time slots.

</p>

</div>

✓

</div>

```
<div class="constraint-card">
```

```
<div class="constraint-icon"><img alt="Icon of a stack of books" data-bbox="412 151 435 170"/></div>
```

```
<div>
```

```
<h3>Weekly Hours</h3>
```

```
<p>
```

```
    Every subject receives its  
    required number of periods.
```

```
</p>
```

```
</div>
```

```
<span class="constraint-status">✔</span>
```

```
</div>
```

```
<div class="constraint-card">
```

```
<div class="constraint-icon"><img alt="Icon of a graduation cap" data-bbox="412 492 435 511"/></div>
```

```
<div>
```

```
<h3>Section Conflict</h3>
```

```
<p>
```

```
    A student section cannot attend  
    two classes at the same time.
```

```
</p>
```

```
</div>
```

```
<span class="constraint-status">✔</span>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
<section class="timetable-section" id="timetable">
```

```
<div class="section-title">
```

AI Generated Timetable

Conflict-free schedule generated using the CSP solver

AI Generated

Status
Not Generated

Generation Time

Total Classes

Conflicts

```
<table class="generated-table">
```

```
  <thead>
```

```
    <tr>
```

```
      <th>Day</th>
```

```
      <th>Time</th>
```

```
      <th>Subject</th>
```

```
      <th>Faculty</th>
```

```
      <th>Room</th>
```

```
      <th>Students</th>
```

```
    </tr>
```

```
  </thead>
```

```
  <tbody id="timetableBody">
```

```
    <tr>
```

```
      <td colspan="6" class="empty-timetable">
```

```
        Click "Generate Timetable" to create a schedule.
```

```
      </td>
```

```
    </tr>
```

```
  </tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
<section class="report-section" id="report">
```

```
  <div class="section-title">
```

```
    <div>
```

```
      <h2><img alt="Bar chart icon" data-bbox="218 792 242 810" /> Conflict Analysis & Performance</h2>
```

```
      <p>Analysis of the generated timetable</p>
```

```
    </div>
```

```
<button class="download-btn" onclick="downloadTimetable()">
```

```
  ⬇ Download Timetable
```

</button>

</div>

<div class="report-grid">

<div class="report-card">

<div class="report-icon">✓</div>

<div>

Constraint Satisfaction

<strong id="satisfactionRate">100%

</div>

</div>

<div class="report-card">

<div class="report-icon">⚡</div>

<div>

Generation Time

<strong id="reportGenerationTime">-

</div>

</div>

<div class="report-card">

<div class="report-icon">⚠</div>

<div>

Conflicts Detected

<strong id="reportConflicts">0

</div>

</div>

<div class="report-card">

<div class="report-icon">🎯</div>

<div>

Solution Quality

<strong id="solutionQuality">Optimal

</div>

</div>

</div>

<div class="analysis-panel">

<h3>🧠 AI Solver Analysis</h3>

<div class="analysis-list">

<div>

Faculty Conflicts

<strong class="passed">✔ 0

</div>

<div>

Room Conflicts

<strong class="passed">✔ 0

</div>

<div>

Room Capacity Violations

<strong class="passed">✔ 0

</div>

<div>

Invalid Time Assignments

<strong class="passed">✔ 0

</div>

<div>

Required Hours Violations

<strong class="passed">✔ 0

</div>

</div>

<div class="success-message">

✔ All hard constraints satisfied. Conflict-free timetable generated successfully.

</div>

</div>

</section>

```
<div class="toast" id="toast">
  <span>✓ </span>
  <p id="toastMessage">Timetable generated successfully!</p>
</div>
...

```

```
</div>
```

```
<script>
```

```
let subjects = [];
```

```
let faculty = [];
```

```
let rooms = [];
```

```
async function addSubject() {
```

```
  let code = document.getElementById("subjectCode").value;
```

```
  let name = document.getElementById("subjectName").value;
```

```
  let teacher = document.getElementById("subjectFaculty").value;
```

```
  let hours = document.getElementById("subjectHours").value;
```

```
  let students = document.getElementById("studentCount").value;
```

```
  if (!code || !name || !teacher || !hours || !students) {
```

```
    showMessage("Please fill all subject details");
```

```
    return;
```

```
  }
```

```
  const response = await fetch("/add_subject", {
```

```
    method: "POST",
```

```
    headers: {
```

```
      "Content-Type": "application/json"
```

```
    },
```

```
    body: JSON.stringify({
```

```
      code: code,
```

```
      name: name,
```

```
      faculty: teacher,
```

```
      hours: hours,
```

```

        students: students
    })
});

const result = await response.json();

if (result.success) {

    document.getElementById("subjectCode").value = "";
    document.getElementById("subjectName").value = "";
    document.getElementById("subjectFaculty").value = "";
    document.getElementById("subjectHours").value = "";
    document.getElementById("studentCount").value = "";

    loadData();

    showMessage("Subject saved to database!");
}
}

function updateSubjectTable() {

    let table = document.getElementById("subjectTable");

    table.innerHTML = "";

    subjects.forEach(function(subject) {

        let row = `
            <tr>
                <td>${subject.code}</td>
                <td>${subject.name}</td>
                <td>${subject.faculty}</td>
                <td>${subject.hours}</td>
                <td>${subject.students}</td>
            </tr>
        `;

        table.innerHTML += row;
    });
}

```

```
}
```

```
async function addFaculty() {  
  
    let name = document.getElementById("facultyName").value;  
  
    let availability =  
        document.getElementById("facultyAvailability").value;  
  
    if (!name) {  
        showMessage("Enter faculty name");  
        return;  
    }  
  
    const response = await fetch("/add_faculty", {  
  
        method: "POST",  
  
        headers: {  
            "Content-Type": "application/json"  
        },  
  
        body: JSON.stringify({  
            name: name,  
            availability: availability  
        })  
    });  
  
    const result = await response.json();  
  
    if (result.success) {  
  
        document.getElementById("facultyName").value = "";  
  
        loadData();  
  
        showMessage("Faculty saved to database!");  
    }  
}
```

```

async function addRoom() {

    let room = document.getElementById("roomNumber").value;
    let capacity = document.getElementById("roomCapacity").value;
    let type = document.getElementById("roomType").value;

    if (!room || !capacity) {
        showMessage("Enter room details");
        return;
    }

    const response = await fetch("/add_room", {

        method: "POST",

        headers: {
            "Content-Type": "application/json"
        },

        body: JSON.stringify({
            room: room,
            capacity: capacity,
            type: type
        })
    });

    const result = await response.json();

    if (result.success) {

        document.getElementById("roomNumber").value = "";
        document.getElementById("roomCapacity").value = "";

        loadData();

        showMessage("Room saved to database!");
    }
}

```

```
async function generateTimetable() {

    const button = document.querySelector(".generate-btn");

    button.innerHTML = "⌚ AI Solving...";
    button.disabled = true;

    try {

        const response = await fetch("/generate", {

            method: "POST",

            headers: {
                "Content-Type": "application/json"
            }

        });

        const result = await response.json();

        if (result.success) {

            button.innerHTML = "✓ Timetable Generated";

            document.getElementById("solverStatus").innerText =
                result.status;

            document.getElementById("generationTime").innerText =
                result.generation_time + " sec";

            document.getElementById("totalClasses").innerText =
                result.timetable.length;

            document.getElementById("conflictCount").innerText =
                "0";
            document.getElementById("satisfactionRate").innerText =
                "100%";
```

```
document.getElementById("reportGenerationTime").innerText =  
    result.generation_time + " sec";
```

```
document.getElementById("reportConflicts").innerText =  
    "0";
```

```
document.getElementById("solutionQuality").innerText =  
    result.status;
```

```
displayTimetable(result.timetable);
```

```
showMessage(  
    "AI timetable generated in "  
    + result.generation_time  
    + " seconds!"  
);
```

```
    } else {
```

```
        button.innerHTML = "⚠ No Solution";
```

```
        showMessage(result.message);
```

```
    }
```

```
} catch (error) {
```

```
    console.error(error);
```

```
    button.innerHTML = "⚠ Solver Error";
```

```
    showMessage(  
        "Unable to connect to AI solver"  
    );
```

```
}
```

```
setTimeout(function() {
```

```
    button.innerHTML = "⚡ Generate Timetable";
```

```

        button.disabled = false;

    }, 2500);
}

function showMessage(message) {

    const toast = document.getElementById("toast");
    const text = document.getElementById("toastMessage");

    text.innerText = message;

    toast.classList.add("show");

    setTimeout(function() {
        toast.classList.remove("show");
    }, 3000);
}

function displayTimetable(timetable) {

    let table = document.getElementById("timetableBody");

    table.innerHTML = "";

    timetable.forEach(function(item) {

        table.innerHTML += `

            <tr>

                <td>
                    <strong>${item.day}</strong>
                </td>

                <td>
                    ${item.period}
                </td>

```

```

        <td>
            <strong>${item.code}</strong><br>
            ${item.subject}
        </td>

        <td>
            ${item.faculty}
        </td>

        <td>
            ${item.room}
        </td>

        <td>
            ${item.students}
        </td>

    </tr>

`;

});

}
function downloadTimetable() {

    let table = document.querySelector(".generated-table");

    let rows = table.querySelectorAll("tr");

    let csv = [];

    rows.forEach(function(row) {

        let cells = row.querySelectorAll("th, td");

        let rowData = [];

        cells.forEach(function(cell) {

```

```

        rowData.push(
            "" + cell.innerText.replace(/"/g, "") + ""
        );

    });

    csv.push(rowData.join(","));

});

let blob = new Blob(
    [csv.join("\n")],
    { type: "text/csv" }
);

let url = URL.createObjectURL(blob);

let link = document.createElement("a");

link.href = url;

link.download = "AI_Timetable.csv";

link.click();

URL.revokeObjectURL(url);
}
async function loadData() {

    const response = await fetch("/data");

    const data = await response.json();

    document.getElementById("subjectCount").innerText =
        data.subjects.length;

    document.getElementById("facultyCount").innerText =
        data.faculty.length;

    document.getElementById("roomCount").innerText =

```

```
    data.rooms.length;

let table = document.getElementById("subjectTable");

table.innerHTML = "";

data.subjects.forEach(function(subject) {

    table.innerHTML += `
        <tr>
            <td>${subject.code}</td>
            <td>${subject.name}</td>
            <td>${subject.faculty}</td>
            <td>${subject.hours}</td>
            <td>${subject.students}</td>
        </tr>
    `;

});

}
loadData();

</script>

</body>
</html>
```

Style.css:

```
* {  
  box-sizing: border-box;  
}  
  
body {  
  margin: 0;  
  font-family: Arial, Helvetica, sans-serif;  
  background: #f5f7fb;  
  color: #20243a;  
}  
  
.sidebar {  
  position: fixed;  
  left: 0;  
  top: 0;  
  width: 245px;  
  height: 100vh;  
  background: #15182b;  
  color: white;  
  padding: 25px 18px;  
  display: flex;  
  flex-direction: column;  
}  
  
.logo {  
  display: flex;  
  align-items: center;  
  gap: 12px;  
  padding: 5px 10px 35px;  
}  
  
.logo-icon {  
  width: 45px;  
  height: 45px;  
  border-radius: 12px;  
  background: linear-gradient(135deg, #7657ff, #a56cff);  
  display: flex;  
  align-items: center;  
  justify-content: center;
```

```
font-size: 23px;  
}
```

```
.logo h2 {  
margin: 0;  
font-size: 18px;  
}
```

```
.logo span {  
font-size: 11px;  
color: #9da1b8;  
}
```

```
.menu {  
display: flex;  
flex-direction: column;  
gap: 8px;  
}
```

```
.menu-item {  
text-decoration: none;  
color: #aeb2c8;  
padding: 13px 15px;  
border-radius: 10px;  
display: flex;  
gap: 13px;  
align-items: center;  
font-size: 14px;  
transition: 0.3s;  
}
```

```
.menu-item:hover,  
.menu-item.active {  
background: #292d4b;  
color: white;  
}
```

```
.menu-item span {  
font-size: 18px;  
}
```

```
.sidebar-bottom {  
margin-top: auto;  
padding: 15px;  
color: #aeb2c8;  
font-size: 13px;  
display: flex;  
align-items: center;  
gap: 8px;  
}
```

```
.status-dot,  
.green-dot {  
width: 8px;  
height: 8px;  
border-radius: 50%;  
background: #2ed573;  
display: inline-block;  
}
```

```
.main {  
margin-left: 245px;  
padding: 30px 45px;  
}
```

```
.topbar {  
display: flex;  
justify-content: space-between;  
align-items: center;  
margin-bottom: 25px;  
}
```

```
.welcome {  
font-size: 11px;  
color: #858ba5;  
letter-spacing: 1.5px;  
margin: 0 0 6px;  
}
```

```
.topbar h1 {
```

```
margin: 0;
font-size: 27px;
}
```

```
.top-status {
background: white;
padding: 10px 16px;
border-radius: 25px;
box-shadow: 0 3px 12px #e4e6ed;
font-size: 13px;
display: flex;
align-items: center;
gap: 8px;
}
```

```
.hero {
min-height: 310px;
border-radius: 22px;
background: linear-gradient(120deg, #28205b, #513b9a);
color: white;
display: flex;
justify-content: space-between;
overflow: hidden;
padding: 42px 50px;
position: relative;
box-shadow: 0 15px 35px #d9d4ee;
}
```

```
.hero-content {
max-width: 650px;
position: relative;
z-index: 2;
}
```

```
.ai-badge {
display: inline-block;
background: #ffffff1f;
border: 1px solid #ffffff35;
padding: 7px 13px;
border-radius: 20px;
```

```
font-size: 11px;  
letter-spacing: 1px;  
}
```

```
.hero h1 {  
font-size: 37px;  
line-height: 1.15;  
margin: 18px 0 12px;  
}
```

```
.hero h1 span {  
color: #c6b5ff;  
}
```

```
.hero p {  
color: #d5d0e8;  
line-height: 1.7;  
font-size: 14px;  
max-width: 590px;  
}
```

```
.generate-btn {  
margin-top: 12px;  
padding: 13px 24px;  
border: none;  
border-radius: 10px;  
background: white;  
color: #46338d;  
font-weight: bold;  
cursor: pointer;  
font-size: 14px;  
transition: 0.3s;  
}
```

```
.generate-btn:hover {  
transform: translateY(-2px);  
box-shadow: 0 8px 20px #00000030;  
}
```

```
.generate-btn.disabled {
```

```
opacity: 0.8;  
cursor: wait;  
}
```

```
.hero-visual {  
width: 300px;  
position: relative;  
}
```

```
.brain-circle {  
width: 145px;  
height: 145px;  
background: #ffffff18;  
border: 1px solid #ffffff30;  
border-radius: 50%;  
position: absolute;  
right: 50px;  
top: 40px;  
display: flex;  
align-items: center;  
justify-content: center;  
font-size: 65px;  
}
```

```
.floating-card {  
position: absolute;  
background: #ffffffea;  
color: #29234c;  
padding: 10px 14px;  
border-radius: 9px;  
font-size: 11px;  
box-shadow: 0 8px 20px #00000020;  
}
```

```
.card-one {  
top: 20px;  
right: 0;  
}
```

```
.card-two {
```

```
bottom: 55px;  
right: 10px;  
}
```

```
.card-three {  
bottom: 5px;  
left: 10px;  
}
```

```
.stats {  
display: grid;  
grid-template-columns: repeat(4, 1fr);  
gap: 18px;  
margin: 25px 0;  
}
```

```
.stat-card {  
background: white;  
border-radius: 15px;  
padding: 20px;  
display: flex;  
align-items: center;  
gap: 15px;  
box-shadow: 0 4px 15px #e6e8ef;  
transition: 0.3s;  
}
```

```
.stat-card:hover {  
transform: translateY(-4px);  
}
```

```
.stat-icon {  
width: 48px;  
height: 48px;  
border-radius: 12px;  
display: flex;  
align-items: center;  
justify-content: center;  
font-size: 21px;  
}
```

```
.purple {  
background: #eeeaff;  
}
```

```
.blue {  
background: #e5f2ff;  
}
```

```
.orange {  
background: #fff0df;  
}
```

```
.green {  
background: #e5faef;  
}
```

```
.stat-card p {  
margin: 0;  
color: #858ba5;  
font-size: 12px;  
}
```

```
.stat-card h2 {  
margin: 5px 0;  
font-size: 25px;  
}
```

```
.stat-card small {  
color: #9da1b8;  
font-size: 10px;  
}
```

```
.dashboard-grid {  
display: grid;  
grid-template-columns: 1.4fr 1fr;  
gap: 20px;  
}
```

```
.panel {
```

```
background: white;
padding: 25px;
border-radius: 17px;
box-shadow: 0 4px 15px #e6e8ef;
}
```

```
.panel-header {
display: flex;
justify-content: space-between;
align-items: center;
}
```

```
.panel-header h2 {
margin: 0;
font-size: 17px;
}
```

```
.panel-header p {
margin: 5px 0;
color: #9297ac;
font-size: 11px;
}
```

```
.success-badge {
background: #e8faef;
color: #20a85a;
padding: 6px 10px;
border-radius: 15px;
font-size: 10px;
font-weight: bold;
}
```

```
.progress-container {
margin: 25px 0;
}
```

```
.progress-info {
display: flex;
justify-content: space-between;
font-size: 12px;
}
```

```
margin-bottom: 9px;
}
```

```
.progress-info strong {
color: #6b4fe8;
}
```

```
.progress-bar {
height: 9px;
background: #eeeeF5;
border-radius: 10px;
overflow: hidden;
}
```

```
.progress {
height: 100%;
width: 100%;
background: linear-gradient(90deg, #6b4fe8, #a56cff);
border-radius: 10px;
transition: width 1s;
}
```

```
.constraint-list {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 12px;
}
```

```
.constraint-list div {
background: #f8f9fc;
padding: 11px;
border-radius: 8px;
font-size: 11px;
color: #60667d;
}
```

```
.constraint-list span {
color: #22b965;
font-weight: bold;
margin-right: 5px;
```

```
}
```

```
.constraint-list strong {  
  float: right;  
  color: #22b965;  
}
```

```
.solver-status {  
  margin-top: 20px;  
  background: #f7f6ff;  
  padding: 15px;  
  border-radius: 12px;  
  display: flex;  
  align-items: center;  
  gap: 12px;  
}
```

```
.solver-icon {  
  width: 43px;  
  height: 43px;  
  border-radius: 10px;  
  background: #e9e4ff;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 21px;  
}
```

```
.solver-status h3 {  
  margin: 0;  
  font-size: 13px;  
}
```

```
.solver-status p {  
  margin: 4px 0 0;  
  font-size: 10px;  
  color: #9398ae;  
}
```

```
.ready {
```

```
margin-left: auto;
color: #20ad5b;
font-size: 9px;
font-weight: bold;
}
```

```
.solver-info {
display: grid;
grid-template-columns: 1fr 1fr;
margin-top: 15px;
gap: 10px;
}
```

```
.solver-info div {
background: #f8f9fc;
padding: 12px;
border-radius: 9px;
}
```

```
.solver-info span {
display: block;
font-size: 10px;
color: #9297ac;
margin-bottom: 5px;
}
```

```
.solver-info strong {
font-size: 11px;
}
```

```
.quick-actions {
background: white;
padding: 22px 25px;
margin-top: 20px;
border-radius: 17px;
box-shadow: 0 4px 15px #e6e8ef;
display: flex;
justify-content: space-between;
align-items: center;
}
```

```
.quick-actions h2 {  
margin: 0;  
font-size: 17px;  
}
```

```
.quick-actions p {  
margin: 5px 0;  
color: #9297ac;  
font-size: 11px;  
}
```

```
.actions {  
display: flex;  
gap: 10px;  
}
```

```
.actions button {  
border: 1px solid #e2e4ec;  
background: white;  
padding: 11px 15px;  
border-radius: 8px;  
cursor: pointer;  
font-size: 11px;  
color: #4d5268;  
transition: 0.3s;  
}
```

```
.actions button:hover {  
border-color: #7657ff;  
color: #7657ff;  
}
```

```
.toast {  
position: fixed;  
right: 30px;  
bottom: 30px;  
background: #22263d;  
color: white;  
padding: 14px 20px;
```

```
border-radius: 10px;
display: flex;
gap: 10px;
align-items: center;
transform: translateY(100px);
opacity: 0;
transition: 0.4s;
box-shadow: 0 10px 30px #00000030;
}
```

```
.toast.show {
transform: translateY(0);
opacity: 1;
}
```

```
.toast span {
color: #2ed573;
font-weight: bold;
}
```

```
.toast p {
margin: 0;
font-size: 12px;
}
```

```
@media (max-width: 1000px) {
```

```
  ``
```

```
.sidebar {
  width: 190px;
}
```

```
.main {
  margin-left: 190px;
  padding: 25px;
}
```

```
.hero-visual {
  display: none;
}
```

```
.stats {
  grid-template-columns: 1fr 1fr;
}

.dashboard-grid {
  grid-template-columns: 1fr;
}

}

.data-section {
  margin-top: 25px;
}

.section-title {
  margin-bottom: 18px;
}

.section-title h2 {
  margin: 0;
  font-size: 20px;
}

.section-title p {
  color: #9297ac;
  font-size: 12px;
}

.data-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 18px;
}

.data-card {
  background: white;
  padding: 22px;
  border-radius: 15px;
  box-shadow: 0 4px 15px #e6e8ef;
```

```
}
```

```
.data-card h3 {  
  margin-top: 0;  
  font-size: 15px;  
}
```

```
.data-card input,  
.data-card select {  
  width: 100%;  
  padding: 11px;  
  margin: 7px 0;  
  border: 1px solid #e1e3eb;  
  border-radius: 7px;  
  outline: none;  
  font-size: 12px;  
}
```

```
.data-card input:focus,  
.data-card select:focus {  
  border-color: #7657ff;  
}
```

```
.input-row {  
  display: flex;  
  gap: 8px;  
}
```

```
.data-card button {  
  width: 100%;  
  padding: 11px;  
  margin-top: 8px;  
  border: none;  
  border-radius: 7px;  
  background: #7657ff;  
  color: white;  
  cursor: pointer;  
}
```

```
.data-card button:hover {
```

```
background: #6044d8;  
}
```

```
.table-panel {  
  background: white;  
  margin-top: 20px;  
  padding: 22px;  
  border-radius: 15px;  
  box-shadow: 0 4px 15px #e6e8ef;  
}
```

```
.table-header h3 {  
  margin: 0;  
}
```

```
.table-header p {  
  color: #9297ac;  
  font-size: 11px;  
}
```

```
table {  
  width: 100%;  
  border-collapse: collapse;  
  margin-top: 15px;  
}
```

```
th {  
  background: #f6f5ff;  
  color: #555a70;  
  padding: 12px;  
  text-align: left;  
  font-size: 11px;  
}
```

```
td {  
  padding: 12px;  
  border-bottom: 1px solid #eeeef4;  
  font-size: 11px;  
}
```

```
@media (max-width: 900px) {  
  .data-grid {  
    grid-template-columns: 1fr;  
  }  
}  
.slot-info {  
  color: #9297ac;  
  font-size: 11px;  
  line-height: 1.6;  
}  
  
.slot-summary {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 6px;  
  margin: 15px 0;  
}  
  
.slot-summary div {  
  background: #f7f6ff;  
  padding: 10px 5px;  
  text-align: center;  
  border-radius: 7px;  
}  
  
.slot-summary strong {  
  display: block;  
  color: #7657ff;  
  font-size: 18px;  
}  
  
.slot-summary span {  
  color: #9297ac;  
  font-size: 8px;  
}  
.constraints-section {  
  margin-top: 25px;  
}  
  
.constraint-grid {
```

```
display: grid;
grid-template-columns: 1fr 1fr;
gap: 15px;
}

.constraint-card {
  background: white;
  padding: 18px;
  border-radius: 13px;
  box-shadow: 0 4px 15px #e6e8ef;
  display: flex;
  align-items: center;
  gap: 14px;
}

.constraint-icon {
  width: 45px;
  height: 45px;
  background: #f0edff;
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 20px;
  flex-shrink: 0;
}

.constraint-card h3 {
  margin: 0;
  font-size: 13px;
}

.constraint-card p {
  margin: 5px 0 0;
  color: #9297ac;
  font-size: 10px;
  line-height: 1.5;
}

.constraint-status {
```

```
margin-left: auto;
width: 25px;
height: 25px;
background: #e6faef;
color: #20ad5b;
border-radius: 50%;
display: flex;
justify-content: center;
align-items: center;
font-size: 12px;
}
```

```
@media (max-width: 900px) {
```

```
  .constraint-grid {
    grid-template-columns: 1fr;
  }
```

```
}
```

```
.timetable-section {
  margin-top: 25px;
}
```

```
.timetable-panel {
  background: white;
  padding: 22px;
  border-radius: 15px;
  box-shadow: 0 4px 15px #e6e8ef;
}
```

```
.timetable-info {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 12px;
  margin-bottom: 20px;
}
```

```
.timetable-info div {
  background: #f7f6ff;
  padding: 15px;
```

```
    border-radius: 10px;
}

.timetable-info span {
    display: block;
    color: #9297ac;
    font-size: 10px;
    margin-bottom: 5px;
}

.timetable-info strong {
    font-size: 16px;
}

.table-wrapper {
    overflow-x: auto;
}

.generated-table {
    width: 100%;
    border-collapse: collapse;
}

.generated-table th {
    background: #7657ff;
    color: white;
    padding: 13px;
    text-align: left;
    font-size: 11px;
}

.generated-table td {
    padding: 13px;
    border-bottom: 1px solid #eeeeF4;
    font-size: 11px;
}

.generated-table tr:hover {
    background: #faf9ff;
}
```

```
.empty-timetable {
  text-align: center;
  color: #9297ac;
  padding: 35px !important;
}

@media (max-width: 900px) {

  .timetable-info {
    grid-template-columns: 1fr 1fr;
  }

}

.report-section {
  margin-top: 25px;
}

.report-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 15px;
}

.report-card {
  background: white;
  padding: 18px;
  border-radius: 13px;
  box-shadow: 0 4px 15px #e6e8ef;
  display: flex;
  align-items: center;
  gap: 13px;
}

.report-icon {
  width: 42px;
  height: 42px;
  background: #f0edff;
  border-radius: 10px;
  display: flex;
```

```
    justify-content: center;
    align-items: center;
    font-size: 19px;
}
```

```
.report-card span {
    display: block;
    color: #9297ac;
    font-size: 10px;
    margin-bottom: 5px;
}
```

```
.report-card strong {
    font-size: 18px;
}
```

```
.analysis-panel {
    background: white;
    margin-top: 18px;
    padding: 22px;
    border-radius: 15px;
    box-shadow: 0 4px 15px #e6e8ef;
}
```

```
.analysis-panel h3 {
    margin-top: 0;
    font-size: 15px;
}
```

```
.analysis-list {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 10px;
}
```

```
.analysis-list div {
    padding: 12px;
    background: #f8f8fc;
    border-radius: 8px;
    display: flex;
```

```
    justify-content: space-between;
}
```

```
.analysis-list span {
    font-size: 11px;
    color: #555a70;
}
```

```
.passed {
    color: #20ad5b;
}
```

```
.success-message {
    margin-top: 18px;
    padding: 13px;
    background: #eafaf0;
    color: #168a46;
    border-radius: 8px;
    font-size: 11px;
}
```

```
@media (max-width: 900px) {
```

```
    .report-grid {
        grid-template-columns: 1fr 1fr;
    }
```

```
    .analysis-list {
        grid-template-columns: 1fr;
    }
```

```
}
```

```
.download-btn {
    margin-left: auto;
    padding: 10px 16px;
    border: none;
    border-radius: 8px;
    background: #7657ff;
    color: white;
    cursor: pointer;
```

```
    font-size: 11px;  
}
```

```
.download-btn:hover {  
    opacity: 0.9;  
}
```